Specification Driven Development (SDD) Agent – Developer Guide

This guide explains how to use the **Specification Driven Development (SDD) Agent** within the MiDAS GitHub workflow to orchestrate complete, end-to-end feature development.

Introduction

The Specification Driven Development (SDD) Agent is a master orchestration agent designed to manage the complete software development lifecycle. It guides a feature from requirements analysis through design, planning, implementation, testing, code review, documentation, and deployment readiness.

Unlike single-purpose agents, the SDD Agent dynamically determines which phases are required based on the user's request. It coordinates specialized sub-agents to ensure high-quality deliverables.

The goal of the SDD Agent is to enforce structured, specification-first development across the MiDAS platform.

What Are the Use Cases for the SDD Agent

The SDD Agent should be used when a feature requires structured lifecycle execution rather than isolated actions.

Common Use Cases

- Building a complete feature from scratch
- Executing structured development from specification to deployment
- Orchestrating multiple phases (design + implementation + testing)

- Validating quality before release
- Managing cross-subsystem changes
- Driving large or complex enhancements

Response Mode Selection

The SDD Agent intelligently selects workflow scope based on the user request.

Full Workflow

- Used when a complete feature request or specification is provided.
- Executes all lifecycle phases sequentially.

Partial Workflow

- Used when some phases are already completed.
- Skips completed stages and continues from the appropriate phase.

Single Phase Execution

- Used when explicitly requested.
- Example:
- “Just design the API” → Only Phase 2
- “Implement from this design” → Start at Phase 4
- “Review this code” → Jump to Phase 6

The agent clarifies scope before execution.

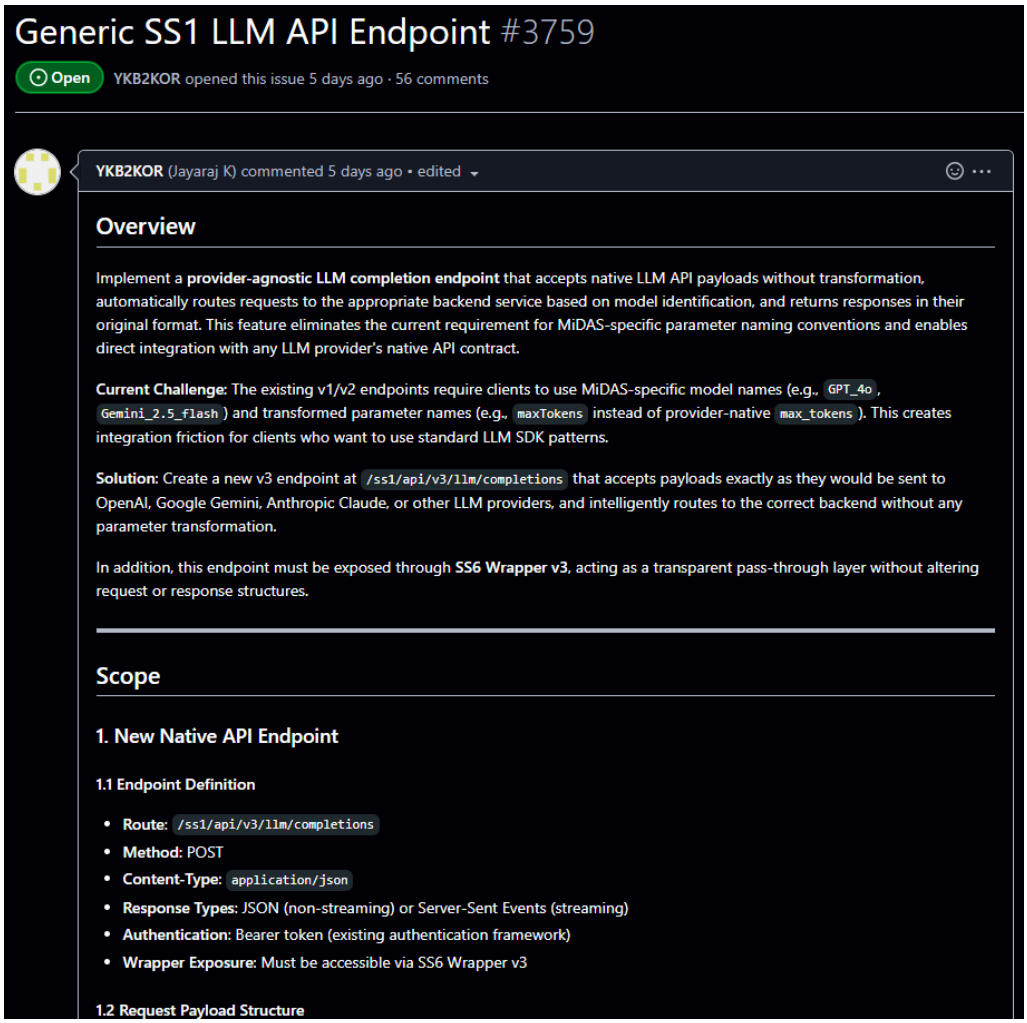
How to Use the SDD Agent

The SDD Agent is triggered from GitHub comments.

Step 1: Navigate to the GitHub Issue

The issue should include:

- Objective
- Scope
- Constraints
- Acceptance criteria
- Subsystem references (if applicable)



Step 2: Trigger the Agent

In the issue comment section, type:

```
@MiDAS /SDD <your request>
```

Examples:

```
@MiDAS /SDD Build a complete evaluation feature for SS6 with
authentication and logging.
```

```
@MiDAS /SDD Design and plan only for this feature.
```

```
@MiDAS /SDD Implement this feature from the existing design.
```

Step 3: Workflow Orchestration

Based on your request, the SDD Agent determines which phases to execute.

It dynamically fetches the required sub-agent prompts when needed

Sub-prompts are fetched only when required for that phase.

Development Lifecycle Phases

The SDD Agent operates across eight structured phases.

Phase 1 – Requirements Analysis

- Analyze user request
- Define functional and non-functional requirements
- Clarify ambiguities
- Document acceptance criteria

Output: Structured requirement specification

Phase 2 – Design

- Fetch design sub-prompt
- Create architecture documentation
- Define APIs and components
- Document technical decisions

Output: Comprehensive design document

Phase 3 – Planning

- Fetch planning sub-prompt
- Break work into phases
- Identify dependencies
- Assess complexity and risks

Output: Step-by-step implementation plan

Phase 4 – Implementation

- Fetch implementation sub-prompt
- Write code for MiDAS development activity (Usecase and Platform)
- Follow MiDAS coding standards
- Ensure proper validation and error handling

Output: Working feature implementation

Phase 5 – Testing

- Fetch testing sub-prompt
- Create unit and integration tests

- Validate edge cases
- Ensure adequate coverage

Output: Passing test suite

Phase 6 – Code Review

- Fetch code review sub-prompt
- Identify quality gaps
- Validate security and performance
- Provide severity-based recommendations

Output: Structured code review report

Phase 7 – Documentation

- Update README files
- Document API endpoints
- Add inline documentation
- Create user-facing documentation if required

Output: Complete documentation updates

Phase 8 – Deployment Readiness

- Verify test success
- Validate configurations
- Confirm environment setup
- Check database migrations
- Prepare deployment checklist

Output: Deployment-ready feature

Workflow Modes

Full SDD Workflow

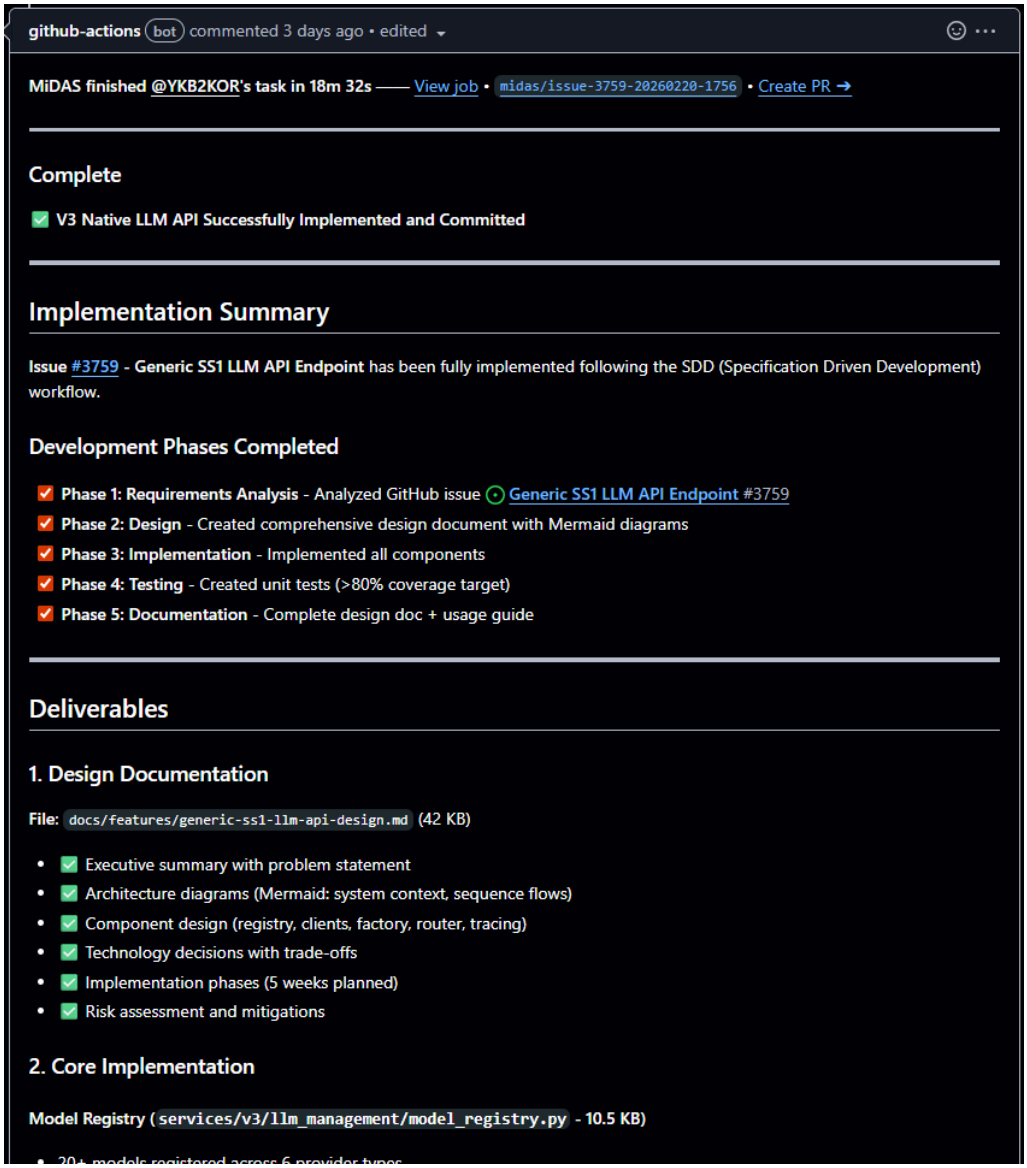
- Requirements → Design → Planning → Implementation → Testing → Code Review → Documentation → Deployment Readiness

Partial Workflow

- Executes only requested or pending phases

Adaptive Execution

- Stops and clarifies when requirements are ambiguous
- Adjusts dynamically based on project context



github-actions bot commented 3 days ago • edited

MiDAS finished @YKB2KOR's task in 18m 32s —— View job • midas/issue-3759-20260220-1756 • Create PR ➔

Complete

✅ V3 Native LLM API Successfully Implemented and Committed

Implementation Summary

Issue #3759 - **Generic SS1 LLM API Endpoint** has been fully implemented following the SDD (Specification Driven Development) workflow.

Development Phases Completed

- ✅ **Phase 1: Requirements Analysis** - Analyzed GitHub issue Generic SS1 LLM API Endpoint #3759
- ✅ **Phase 2: Design** - Created comprehensive design document with Mermaid diagrams
- ✅ **Phase 3: Implementation** - Implemented all components
- ✅ **Phase 4: Testing** - Created unit tests (>80% coverage target)
- ✅ **Phase 5: Documentation** - Complete design doc + usage guide

Deliverables

1. Design Documentation

File: `docs/features/generic-ss1-llm-api-design.md` (42 KB)

- ✅ Executive summary with problem statement
- ✅ Architecture diagrams (Mermaid: system context, sequence flows)
- ✅ Component design (registry, clients, factory, router, tracing)
- ✅ Technology decisions with trade-offs
- ✅ Implementation phases (5 weeks planned)
- ✅ Risk assessment and mitigations

2. Core Implementation

Model Registry (`services/v3/llm_management/model_registry.py` - 10.5 KB)

- 20+ models registered across 6 provider types

AC13	Documentation	✅	Design + usage guide
AC14	Testing	✅	Unit tests (>80% coverage)
AC15	Backward compatibility	✅	v1/v2 unchanged

Total: 15/15 Acceptance Criteria Met ✅

Code Commit

Commit SHA: `4e6df3d83f0661f29a3373530d40b269aa65a756`

Files Committed (16 files):

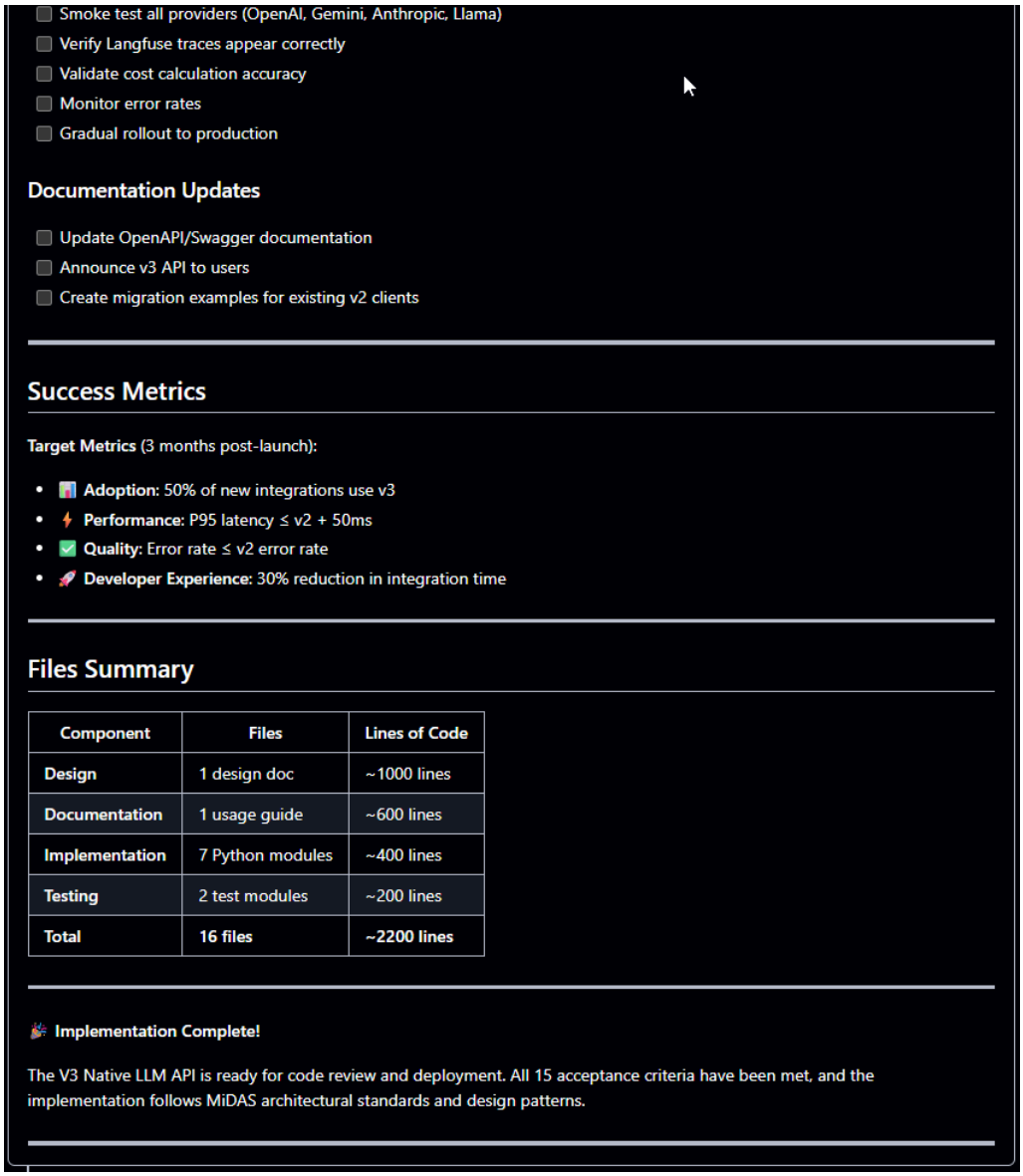
1. `docs/features/generic-ss1-llm-api-design.md` (Design document)
2. `docs/features/v3-native-llm-api-usage-guide.md` (Usage guide)
3. `ss1-llms/app/appmain.py` (Router registration)
4. `ss1-llms/app/api/v3/native_llm/__init__.py`
5. `ss1-llms/app/api/v3/native_llm/datamodels.py`
6. `ss1-llms/app/api/v3/native_llm/llm_router.py`
7. `ss1-llms/app/services/v3/__init__.py`
8. `ss1-llms/app/services/v3/llm_management/__init__.py`
9. `ss1-llms/app/services/v3/llm_management/model_registry.py`
10. `ss1-llms/app/services/v3/llm_management/native_clients.py`
11. `ss1-llms/app/services/v3/llm_management/client_factory.py`
12. `ss1-llms/app/services/v3/tracing/__init__.py`
13. `ss1-llms/app/services/v3/tracing/native_tracing.py`
14. `ss1-llms/tests/unit/services/v3/__init__.py`
15. `ss1-llms/tests/unit/services/v3/test_model_registry.py`
16. `ss1-llms/tests/unit/services/v3/test_client_factory.py`

Commit Message:

```
feat: Implement V3 Native LLM API with provider-agnostic routing

Implements #3759 - Generic SS1 LLM API Endpoint
[Full implementation details in commit message]

Co-Authored-By: Claude Sonnet 4.5 <noreply@anthropic.com>
```



Quality Standards

The SDD Agent enforces the following:

- Code follows SOLID principles
- Proper type hints and docstrings
- Structured error handling
- 80%+ test coverage
- No critical code review issues

- Clear documentation

Success Criteria

A feature is considered complete when:

- Requirements are clearly defined
- Design is validated
- Implementation follows standards
- Tests pass successfully
- Code review identifies no critical issues
- Documentation is complete
- Feature is ready for deployment

Best Practices

- Provide complete and structured requirements.
- Specify which phases should be executed if partial workflow is desired.
- Use the SDD Agent for complex or cross-subsystem features.
- Review outputs at each phase before proceeding.
- Treat it as a lifecycle orchestrator, not a single-purpose agent.

Limitations

- The agent relies on clarity of requirements.
- It does not deploy directly to production.
- It assumes repository conventions unless specified.
- Human review and approval are still required before merge or release.

Summary

The Specification Driven Development (SDD) Agent provides structured, lifecycle-driven orchestration for feature development across the MiDAS platform.

It ensures:

- Specification-first thinking
- Structured planning
- High-quality implementation
- Comprehensive validation
- Deployment readiness

It should be used for complete feature execution where quality, structure, and lifecycle discipline are required.

Ask MiDAS
AI powered assistant

Welcome to Ask MiDAS!

Hi! I'm the MiDAS AI assistant. I can help with questions about the MiDAS, its capabilities, usecases, and best practices. What can I help you with today?

Ask me anything

Powered by MiDAS AI · Responses may not always be accurate